Task 2 - Boot-time PQC Protection

PSoC Control C3M6 (KIT_PSC3M6_EVAL) - Edge Protect Bootloader verifies the application with ML-DSA-87

1. Summary

The Edge Protect Bootloader (EPBL) was reconfigured from ECDSA-P384 to **ML-DSA-87** (FIPS 204, security category 5) in **HashML-DSA** mode with a **SHA-512** pre-hash, matching the CNSA 2.0 algorithm profile. The application is authenticated by EPBL on every boot and before and after every update. All 4 positive tests passed and all 11 negative tests were rejected on hardware, including an ECDSA-signed image (no silent classical fallback), a wrong pre-hash, an unsupported ML-DSA parameter set, a malformed signature TLV and an unauthorized rollback. ML-DSA-87 verification is **faster** than the software ECDSA-P384 baseline (61 ms vs 170 ms); the cost is EPBL flash (+11.4 KB), boot RAM (+14.5 KB peak) and 4.6 KB extra signature metadata per image. The BootROM stage stays classical (ECDSA, immutable) and is not enabled on the shared DEVELOPMENT board.

Area	Result	Key figures (ECDSA-P384 -> ML-DSA-87)
Positive tests	4/4 PASS (boot, restore, valid update, reboot after update)	reset-to-app 266 -> 146 ms
Negative tests	11/11 REJECTED	verify 170 -> 61 ms
Resources	fits: EPBL 43 % of its 108 KB slot, peak boot RAM 25 KB of 54 KB	EPBL flash 35.9 -> 47.2 KB; peak RAM 10.7 -> 25.2 KB
Known limitation	older signed image written directly to the primary slot via debug boots (L1)	needs HW NV counter (eFuse) + closed DAP

2. Configuration (ECC baseline vs PQC)

Item	Task 1 baseline	Task 2 PQC
EPBL validation_key_type	ECDSA-384	**ML-DSA-87**
Signature mode	ECDSA over SHA-384 image hash	HashML-DSA (FIPS 204 sec. 5.4) over SHA-512 image hash, wolfCrypt wc_dilithium_verify_ctx_hash
Image hash TLV / key-hash TLV	SHA-384 48 B / 48 B	SHA-512 64 B / 64 B
Signature TLV	0x22 ECDSA_SIG, 103 B (DER)	0x105 ML-DSA-87, 4,627 B (raw)
TLV area per image	215 B	4,771 B
Validation public key	P-384, 120 B DER, in EPBL flash	ML-DSA-87, 2,592 B raw, in EPBL flash (C array)
Image format	MCUboot, header 0x400, overwrite mode, slot 128 KB	unchanged
Rollback	sw_downgrade_prev = true	unchanged (hw_rollback_prot = false, eFuse)
Config files	feature_config_ECDSA-384_baseline.json	feature_config_ML-DSA-87.json (active)

Signer change. The CE243041 signing step (configs/prebuild.py) was hard-wired to SHA-256 for ML-DSA, while the EPBL Makefile defaults ML-DSA-87 to SHA-512 - left at defaults, the two disagree and the image never boots. A *--sha* option / Makefile variable *IMAGE_SHA* was added; both sides are now set explicitly to SHA-512. SHA-256 was rejected for the CNSA 2.0 profile (CNSA 2.0 allows SHA-384/512; a 256-bit pre-hash gives only 128-bit collision resistance, below category 5). Test N9 shows a SHA-256 pre-hash image is refused.

3. Boot chain and the exact authentication points

```
Reset
 +- ROM_BOOT  (immutable, Infineon)   validates SFLASH / FLASH_BOOT          classical
     +- FLASH_BOOT  policy SIMPLE_APP (DEVELOPMENT LCS, no OEM key)
         |   -> launches EPBL WITHOUT authentication   [SECURE_APP would verify EPBL: ECDSA only]
         +- EPBL @0x32000000  (MCUboot + wolfCrypt ML-DSA, key embedded)
             |  A) every boot: primary slot @0x3201B000  SHA-512 hash, key hash, ML-DSA-87
             |  B) update: upgrade slot @0x32040000      ML-DSA-87 + version >= current
             |  C) after copy: primary slot again        ML-DSA-87
             +- application (CE243041 demo; SPDM responder in Task 3/4)
```

BootROM vs EPBL. AN243370 sec. 1.3: BootROM secure boot uses ECDSA (P-256/384/521) and 'for this feature [PQC], Edge Protect Bootloader (EPB) must be used as first OEM application in the boot chain'. The BootROM is therefore not assumed to verify ML-DSA. On this board the BootROM does not authenticate EPBL at all (SIMPLE_APP, Task 1 section 3); enabling it needs an EPSS-signed OEM certificate and an ECDSA OEM_ROT_KEY.

Key provisioning. The ML-DSA-87 public key is converted to a C array at build time (generated/ml_dsa_sign_key.c) and lives in EPBL flash. SFLASH key storage is ECDSA-only (EPBL Makefile refuses ML-DSA with sflash_keys). The key's integrity therefore equals EPBL's integrity: AN243370 sec. 9.3.6 requires closing the debug port in production, and BootROM authentication of EPBL is what anchors it. Key rotation = new EPBL build; images signed for the old key are rejected (N7).

4. Test results on hardware

#	Case (image presented to EPBL)	Expected	Observed (EPBL UART log / readback)	Result
P1	authorized v1.0.0, ML-DSA-87 + SHA-512	boot	validate returned 0x742e, app prints 1.0.0+0	PASS
N1	payload byte flipped @0x3201C000	reject	IMAGE_TLV_SHA is invalid, not valid	REJECTED
N2	MCUboot header: version byte @0x3201B014	reject	IMAGE_TLV_SHA is invalid (header is hashed)	REJECTED
N3	protected TLV byte @0x3202546C	reject	IMAGE_TLV_SHA is invalid	REJECTED
N4	key-hash TLV byte @0x32025559	reject	returned 0x2e - no trusted key matches	REJECTED
N5	byte inside ML-DSA-87 signature @0x32025985	reject	hash OK; ML-DSA verification fails at 120 ms	REJECTED
N6	malformed: signature TLV length 4627 -> 4611	reject	returned 0xffffffff, not valid	REJECTED
N7	image signed with a rogue ML-DSA-87 key	reject	returned 0x2e, app not started	REJECTED
N8	ECDSA-P384 signed image (classical downgrade)	reject	no ECDSA path compiled in; not valid	REJECTED
N9	ML-DSA-87 with SHA-256 pre-hash (non-compliant)	reject	returned 0xffffffff, not valid	REJECTED
N11	image signed with ML-DSA-65 (unsupported parameter set, 3,309 B sig)	reject	returned 0x2e - no ML-DSA-65 key/verifier, not valid	REJECTED
P2	restore authorized v1.0.0	boot	app prints 1.0.0+0	PASS
N10	rollback: correctly signed v0.9.0 in upgrade slot	reject	'insufficient version in secondary slot' at 47 ms; readback before: upgrade hdr 96F3B83D ver 0.9; after: upgrade slot all 0x00, primary ver 1, app 1.0.0+0	REJECTED
P3	valid update v2.0.0+1 in upgrade slot	install	verify upgrade slot (43 -> 112 ms) -> erase -> copy 0x20000 -> verify slot 1 -> app 2.0.0+1	PASS
P4	reboot after update	boot	app prints 2.0.0+1	PASS
L1	limitation: signed v0.9.0 written directly to primary slot via J-Link	-	boots 0.9.0+0 - SW downgrade check only covers the update path	BOOTS

Tampered images are produced from the authorized image by tools/hex_flip.py at addresses printed by tools/mcuboot_info.py; the whole matrix runs with tools/task2_boot_tests.ps1. L1 is shown deliberately: rollback protection against an attacker with flash/debug access needs hw_rollback_prot (eFuse NV counter, irreversible, not enabled on the shared board) plus a closed debug port.

5. Resource and performance comparison (boot-time)

Metric	ECDSA-P384	ML-DSA-87 / SHA-512	Delta	Source
Signature verification	~170 ms	**~61 ms**	-109 ms	EPBL log 59 -> 229 / 59 -> 120 ms
boot_validate_slot (hash + key hash + signature)	~179 ms	~69 ms	-110 ms	EPBL log
Reset -> start of application	~266 ms	**~146 ms**	-120 ms	EPBL log
Reset -> application with update install	~3.80 s	~3.57 s	-0.23 s	erase + copy 128 KB + 2 verifications
EPBL flash (.text + .data)	35,888 B	**47,244 B**	+11,356 B	build report; slot 110,592 B
EPBL static RAM (.data + .bss)	6,948 B	19,044 B	+12,096 B	static MlDsaKey ~12.5 KB
EPBL peak stack (full verification)	3,752 B	**6,120 B**	+2,368 B	J-Link stack painting, 16 KB stack
EPBL heap touched	120 B	120 B	0	painting
Peak boot RAM (static + stack)	~10.7 KB	**~25.2 KB**	+14.5 KB	EPBL RAM region 54 KB
Signature metadata per image	215 B	4,771 B	+4,556 B	both slots; 128 KB slot

Metric	ECDSA-P384	ML-DSA-87 / SHA-512	Delta	Source
Validation key in EPBL	120 B	2,592 B	+2,472 B	flash

ML-DSA-87 verification is faster than ECDSA-P384 here because P-384 runs in software mbedTLS (only P-256 is CryptoLite-accelerated) while ML-DSA verify is NTT arithmetic plus SHAKE (wolfCrypt, VERIFY_SMALL_MEM). Timings are from the Debug EPBL with debug logging, identical for both variants. Correction to Task 1: update install took ~3.8 s reset-to-app (Task 1 quoted only the erase phase).

6. CNSA 2.0 mapping (design constraint, not a certification claim)

Function	ECC baseline	CNSA 2.0	Selected	Status
Firmware signature EPBL -> app	ECDSA-P384	ML-DSA-87 (all uses) or LMS/XMSS (SP 800-208, firmware)	**ML-DSA-87**	PQC
Hash (image + pre-hash)	SHA-384	SHA-384 / SHA-512	**SHA-512**	compliant
BootROM -> EPBL	ECDSA (not enabled)	-	unchanged	classical (immutable ROM)
Rollback / anti-downgrade	SW version check	-	SW check; HW counter documented	partial (L1)

ML-DSA-87 is the only ML-DSA parameter set in CNSA 2.0 (category 5); ML-DSA-44/65 would not meet it. LMS/XMSS are also allowed for firmware signing and are supported by EPBL, but are stateful (AN243370 sec. 9.3.6: state reuse breaks security). ML-DSA-87 was chosen as stateless and identical to the SPDM signature algorithm planned for Task 3, so one PQC family covers both planes. No hybrid ECC+PQC mode is used (challenge rule).

7. Transition from the ECC root of trust and remaining classical dependencies

Stage	Today (board)	Production target
ROM_BOOT / FLASH_BOOT	classical; does not check EPBL (SIMPLE_APP)	SECURE_APP: authenticates EPBL with ECDSA OEM_ROT_KEY - remains classical, ROM cannot change
EPBL -> application	**ML-DSA-87 (PQC)**	ML-DSA-87, optionally HW NV counter
ML-DSA validation key	embedded in EPBL, open DAP	embedded in a BootROM-authenticated EPBL, DAP closed
EPBL update	J-Link only (no EPBL update slot)	EPBL update through the BootROM-authenticated path

Migration path for deployed ECC devices: (1) ship an EPBL build that verifies ML-DSA-87 (this task) - this is a bootloader replacement, so it needs either a factory/service reflash or an ECDSA-authenticated EPBL update path via BootROM; (2) from then on all application images are signed with ML-DSA-87 only and ECDSA images are refused (N8); (3) ECDSA remains only in the immutable BootROM->EPBL step, which is the documented residual classical dependency until silicon with a PQC-capable ROM is available.

Not done / limits: BootROM authentication of EPBL (needs EPSS certificate), HW anti-rollback (eFuse, irreversible), SPDM responder behind EPBL (responder map 288 KB vs 128 KB slot, PAS @0x32048000 overlaps the upgrade slot - addressed in Task 4).

8. Evidence (files in results/)

File	Content
task2_boot/summary_ML-DSA-87.txt, summary_ML-DSA-87_rerun_N11_N10.txt, summary_ML-DSA-87_rerun_N10.txt	test matrix output incl. readbacks (N11 added; N10 re-run with complete boot log)
task2_boot/boot_ML-DSA-87_*.log	EPBL UART logs P1-P4, N1-N11, L1
task2_boot/neg_ML-DSA-87_*.hex, img_ML-DSA-87_*.hex	exact images used
task2_boot/N10_*, P3_* readback txt	flash readback before/after rollback and update
epbl_mldsa87_build.log, epbl_ecdsa384_rebuild_task2.log	EPBL memory reports
epbl_ML-DSA-87.hex	programmed bootloader
TASK2_NOTES.md	working notes, reproduce commands